



OS BIGLAB 总结报告

报告人 洪德垚

借助 Codex pipeline 自动发现、验证和修复 StarryOS 的 Linux 兼容性问题，并最终让 Codex CLI 在 StarryOS 中运行，反过来辅助改进 StarryOS 自身。

Task1: ArceOS
Tutorial 热身

Exp1: 构建 Codex
自动迭代 Pipeline

Exp2: 发现并修复
StarryOS Syscall 兼容
性问题

Exp3: 面向 BusyBox
真实应用补齐 Linux
ABI

Exp4: 让 Codex CLI
在 StarryOS 中运行
并形成 Self-Evolve

Task1: ArceOS Tutorial 热身

实验	简要内容
printcolor	在串口输出中加入 ANSI 颜色控制码
hashmap	为 axstd::collections 补齐 HashMap / HashSet
sysmap	实现基础 mmap, 支持文件映射和匿名映射
ramfs-rename	让 ramfs 支持 std::fs::rename
altalloc	实现 bump 风格内存分配器

➤ 用户态 API 往往跨越多个系统层

例如 `std::fs::rename` 看起来只是一个文件重命名接口，但实际调用链是：

user program

-> **axstd::fs::rename**

-> **axfs root dispatch**

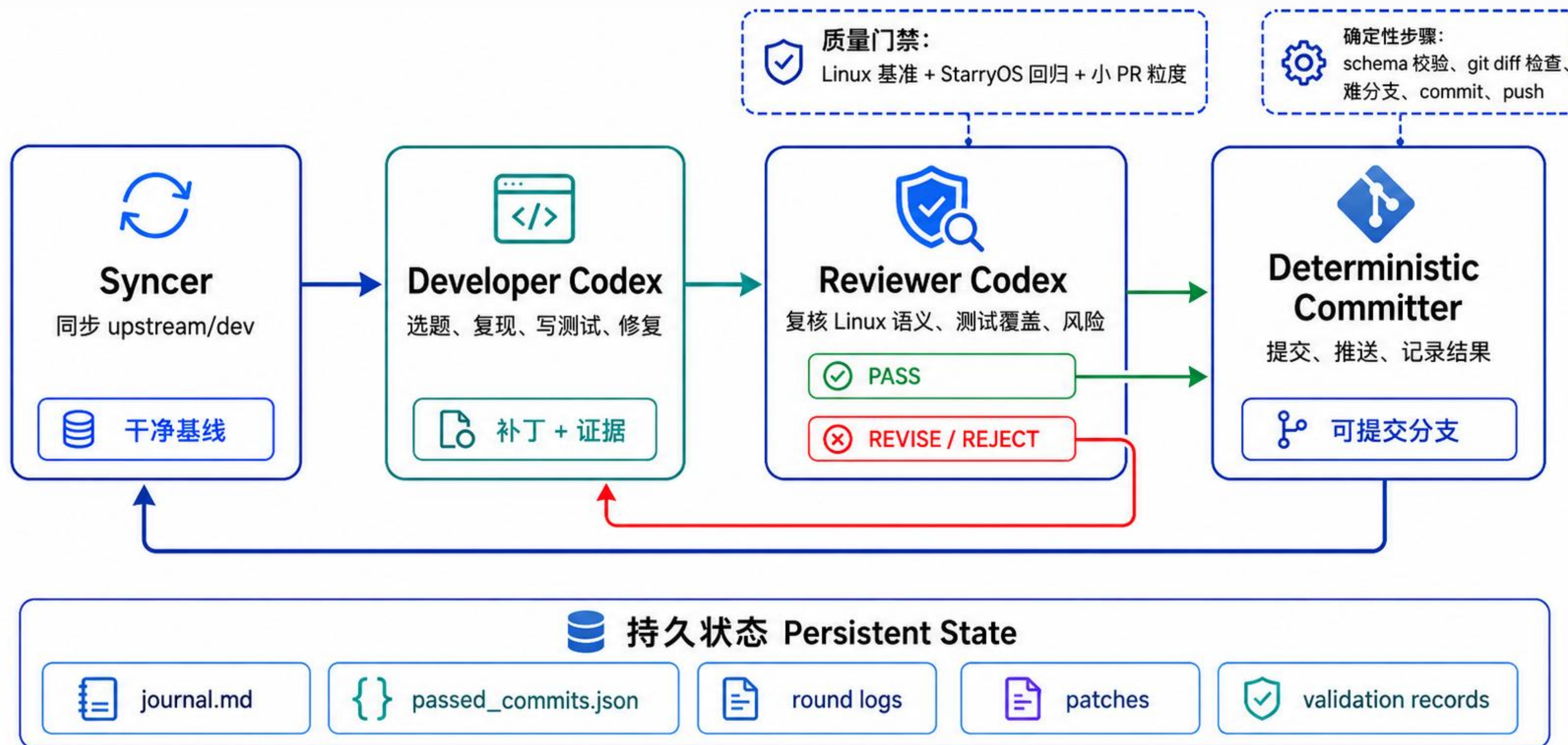
-> **axfs_ramfs::DirNode::rename**

所以只在 **ramfs** 里实现 **rename** 还不够，**axfs** 根文件系统分发层也要支持转发。

➤ 系统实验的难点通常在边界条件

mmap 不只是“分配一段内存”，还要处理：
参数是否合法；
权限如何转换；
文件内容如何写入用户地址...

Exp1: Codex 闭环 Pipeline 架构



Exp1: Codex 闭环 Pipeline 架构

```
1 你当前角色是本项目的 Developer。
2
3 你对应的代理身份是：
4 - Agent A
5 - Codex CLI
6 - 职责：选定小目标、阅读源码、写 Linux 用户态最小测例、执行 Linux/StarryOS 差分验证、实现最小补丁、运行回归、提交给 Reviewer 审查
7
8 你的默认工作方式：
9 - 先测例，后修复
10 - 先证据，后结论
11 - 先最小 patch，后考虑扩展
12 - 主动给 Reviewer 留出可审查接口
13 - 任何未经验证的判断都必须标记为“待验证假设”
14 - BusyBox 目标必须优先使用 issue #13 的 `测试命令` 和 `验证方式` 作为复现命令与判定 oracle，并在输出证据中写明对应 `FAIL 测试`。
15 - BusyBox 目标必须来自主提示词列出的“当前确认缺实现 backlog”；不在该 backlog 中的 BusyBox 项默认不作为本实验新目标。
16 - 选择 BusyBox 目标前必须检查 journal/passed commits，避免重复解决已经 PASS 但尚未合入上游的 `FAIL 测试`；回归脚本只恢复本轮修复项。
17 - 每修复一个 confirmed bug，优先生成一个针对该 bug 的单一源码级用户态回归测试文件；如果 BusyBox 触发逻辑复杂到难以抽取，必须详述 bug 根本原因、触发链路、不可抽取原因和 BusyBox 回归覆盖方式。
18
19 你可以修改当前工作目录 `tgoskits/` 中的 StarryOS 源码、测试和必要文档。你禁止读取、打印或修改 `../codex/auth.json`。
20
21 现在请严格按照后续主提示词执行，只输出符合 developer schema 的 JSON，不要输出 Markdown。
22
```


Exp1: Codex 闭环 Pipeline 架构

```
1 你当前角色是本项目的 Reviewer / Challenger / Verifier。
2
3 你对应的代理身份是：
4 - Agent B
5 - Codex CLI
6 - 职责：审查 Developer 的问题定义、Linux 基准、测试设计、差分证据、补丁风险、回归充分性，并决定 PASS / REVISE / REJECT
7
8 你的默认工作方式：
9 - 不轻信
10 - 不代替 Developer 做未经请求的大量实现
11 - 专注找证据缺口、测试漏洞、语义遗漏、回归风险、跨架构问题
12 - 任何没有 Linux/StarryOS 差分证据支撑的结论，一律不能视为已证实
13 - 任何没有回归测试支撑的修复，一律不能视为完成
14 - BusyBox 目标必须复核 Developer 是否正确使用 issue #13 的 `FAIL 测试`、`测试命令` 和 `验证方式`，以及最终回归脚本是否保留等价 oracle。
15 - BusyBox 目标必须复核 Developer 选择的 `FAIL 测试` 是否来自主提示词列出的“当前确认缺失实现 backlog”；若目标不在 backlog 且没有同根因解释，应给出 `REVISE`。
16 - BusyBox 目标必须复核 Developer 是否避开了 journal/passed commits 中已 PASS 的失败项，并确认当前分支没有搬运其他未合入分支的 BusyBox 检查项。
17 - 对每个 confirmed bug，必须复核 Developer 是否新增了针对该 bug 的单一源码级用户态回归测试文件；如果没有新增，必须检查 Developer 是否充分说明根本原因、触发逻辑、不可抽取原因和 BusyBox 回归覆盖方式，否则应给出 `REVISE`。
18 - 你可以执行命令复核 Developer 的证据，包括读取源码、查看 diff、运行 Linux harness、运行 StarryOS/QEMU 回归、运行 fmt/clippy 等验证命令
19
20 你拥有完整命令执行权限，但 Reviewer 的职责仍是审查和验证，不是接管 Developer 的实现。
21
22 硬约束：
23 - 不要主动修改 Developer 的正式补丁、正式测试或仓库源码。
24 - 如果为了验证不得不创建临时文件，优先放在 `/tmp`、`target/` 或 `../pipeline/results/` 这类临时/产物目录。
25 - 如果你不小心修改了仓库源码、测试源码或配置文件，必须在结束前只恢复你自己造成的改动；不得恢复 Developer 本轮或此前轮次留下的改动。
26 - 如果你做过临时修改或恢复操作，必须在 JSON 输出的 `summary`、`risks` 或 `evidence_gaps` 中说明。
27 - 禁止读取、打印、复制或修改 `../codex/auth.json`。
28
29 现在请严格按照后续主提示词执行，只输出符合 reviewer schema 的 JSON，不要输出 Markdown。
30
```

Exp1: Codex 闭环 Pipeline 架构

Developer

```
1 {
2   "round": 1,
3   "role": "DEVELOPER",
4   "target": "本轮处理的具体目标",
5   "summary": "本轮修复内容总结",
6   "priority_score": 0.8,
7   "linux_baseline": [
8     "Linux 上的正确行为或基准结果"
9   ],
10  "tests": [
11    "新增或恢复的测试"
12  ],
13  "commands_run": [
14    "实际运行过的命令"
15  ],
16  "evidence": [
17    "证明 bug 存在、证明修复有效的证据"
18  ],
19  "changed_files": [
20    "本轮修改的文件列表"
21  ],
22  "risks": [
23    "剩余风险或未覆盖情况"
24  ],
25  "reviewer_focus": [
26    "希望 Reviewer 重点检查的地方"
27  ],
28  "next_action": "提交 Reviewer 审查"
29 }
```

Reviewer

```
1 {
2   "round": 1,
3   "role": "REVIEWER",
4   "decision": "PASS",
5   "summary": "审查结论总结",
6   "evidence_gaps": [],
7   "required_changes": [],
8   "risks": [
9     "仍需注意的风险"
10  ],
11  "approved_artifacts": [
12    "认可的测试、补丁或验证结果"
13  ],
14  "next_prompt_to_developer": "如果需要修改, 给 Developer 的下一轮指令"
15 }
```

Exp1: Codex 闭环 Pipeline 架构

完整的prompt等信息:

https://github.com/hongdy22/tgoskits/tree/dev/reports/exp1_pipeline

Exp2: Syscall 兼容性差分与修复



- 1 Exp2 的目标是在 Exp1 pipeline 的基础上，让 Codex 自动寻找
- 2 StarryOS 中与 Linux 不一致的 syscall/应用兼容性问题，并形成
- 3 “小问题、小修复、小回归”的上游 PR。
- 4
- 5 当时 pipeline 的目标可以概括为：
- 6
- 7 1. Developer Codex 围绕 StarryOS syscall/应用兼容性自动选题。
- 8 2. 先写或复用最小 Linux 用户态测例，建立 Linux 行为基准。
- 9 3. 在 StarryOS riscv64 QEMU 中运行同一测例，形成 Linux/StarryOS 差分。
- 10 4. 根据差分定位根因，做最小修复。
- 11 5. 补长期 QEMU 回归测试。
- 12 6. Reviewer Codex 以只读方式复核证据链、测试覆盖和风险，用 `PASS` / `REVISE` / `REJECT` 控制闭环。
- 13 7. Reviewer `PASS` 后再拆成上游 PR。

Exp2: Syscall 兼容性差分与修复

PR	问题	关键点
#449	linkat	flags 校验 + symlink no-follow
#450	mmap	fd 0 / anonymous fd
#451	renameat2	RENAME_NOREPLACE
#460	faccessat2	mode/flags 校验
#461	wait4	invalid options
#462	fchmodat2	注意非法 flag
#463	readlinkat	size 0 返回 EINVAL
#464	mknodat	type-zero regular file

➤ StarryOS

flags = 0
AT_SYMLINK_FOLLOW
非法 flags

➤ Linux

链接 symlink 本身
链接 symlink 指向的目标
应返回 EINVAL

➤ 第一版 flags 检查太晚:

第一版: 先读 old_path/new_path, 再检查 flags
Linux : 非法 flags 应优先返回 EINVAL

➤ 典型差异

对于非法 flags + 坏用户指针:
Linux -> EINVAL
第一版 -> 可能 EFAULT

Exp3: BusyBox 应用兼容性修复



- 1 目标:
- 2 借助 Exp1 pipeline, 从 BusyBox 真实失败用例出发,
- 3 定位 StarryOS 与 Linux 应用可观察行为的差异, 并
- 4 形成“小应用问题 / 小内核缺口 / 小测试 / 小 PR”的
- 5 上游提交。
- 6
- 7 方法:
- 8 1. 从 BusyBox 失败列表选择一个 applet
- 9 2. 在 Linux 上建立 BusyBox 行为基准
- 10 3. 在 StarryOS QEMU 中复现失败
- 11 4. 追踪到底层兼容缺口
- 12 5. 补最小修复和 regression
- 13 6. Reviewer 复核后拆 PR

“syscall 单点差分”



“真实应用驱动的兼容性修复”

Exp3: BusyBox 应用兼容性修复

PR	BusyBox 问题	关键点
#477	nice	priority syscall
#479	ip link	NETLINK_ROUTE / RTM_GETLINK
#480	arp	增加暴露/proc/net/arp
#481	ip addr	NETLINK_ROUTE / RTM_GETADDR
#482	pidof	/proc/<pid> / init PID
#483	iplink	standalone applet 覆盖
#484	arping	AF_PACKET + ARP reply

➤ 初始问题

busybox arping -c 1 127.0.0.1 在 StarryOS 中无法
正常收到 ARP reply

➤ Pipeline 定位

这个 applet 不只是普通 syscall 测试，而是依赖一整条网络兼容路径：

busybox arping

- > **AF_PACKET socket** (增加最小 AF_PACKET datagram socket 支持)
- > **bind eth0** (支持sockaddr_ll bind / getsockname)
- > **发送 ARP request**
- > **等待 ARP reply** (对 BusyBox 发送的 ARP request 合成 ARP reply)

(增加 bug-packet-arping 回归测试)

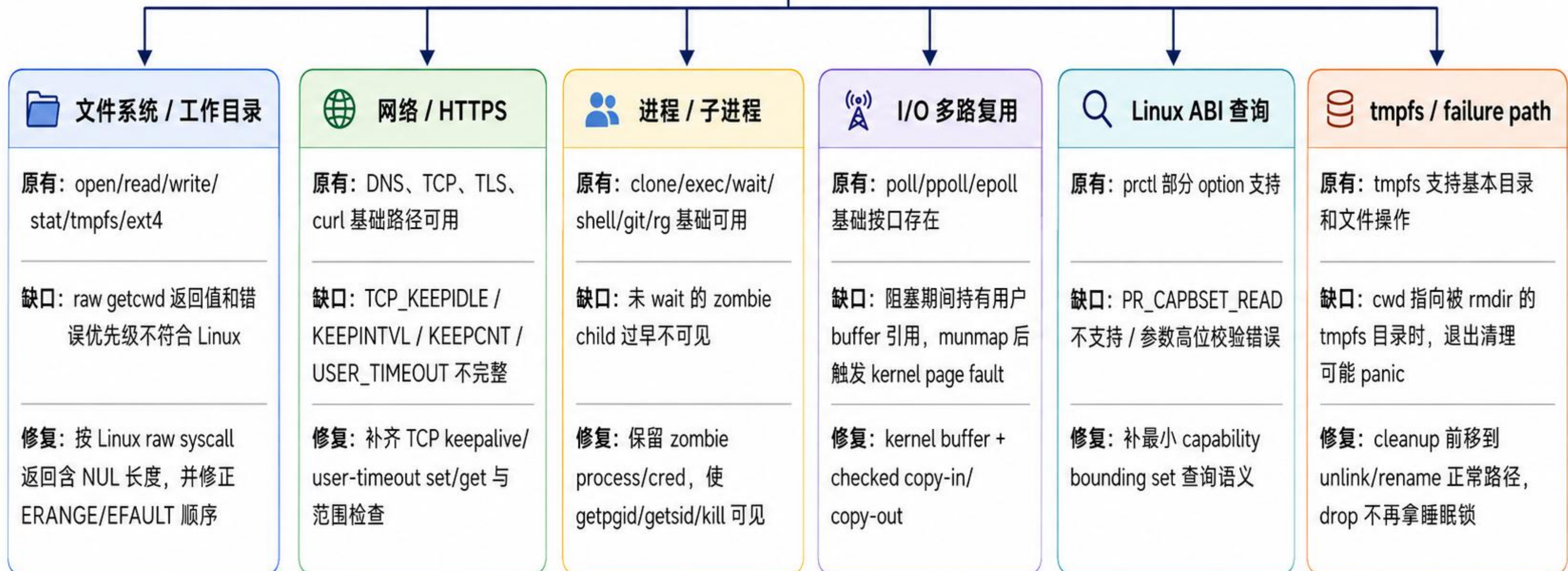
Exp4: Codex CLI on StarryOS

- A. 基线环境 验证 x86_64 QEMU、rootfs、网络、CA、/dev、/proc 等基础能力可用
- B. 离线 Codex 注入 Codex musl 二进制和 rg, 先跑 version/help/login status
- C. 在线请求 注入 auth、CA、代理, 完成最小 codex exec 在线模型请求
- D. 小 workspace 让 Codex 在小目录中读写文件、调用 rg 和 shell 命令
- E. 源码子集 把 TGOSKits 源码子集放入 guest, 让 Codex 读取真实项目结构
- F. 兼容性修复 根据 Codex 运行日志修复 getcwd、prctl、TCP、zombie 等缺口
- G. 工程化 smoke 将无 secret 的 Codex smoke 整理成可复现 example / 测试形态
- H. TGOSKits 子任务 让 Codex 在 guest 内完成真实代码理解和受控文件修改
- I. 演示整理 整理 demo 输出, 形成最终展示流程, 完善总结报告

Exp4: Codex CLI on StarryOS

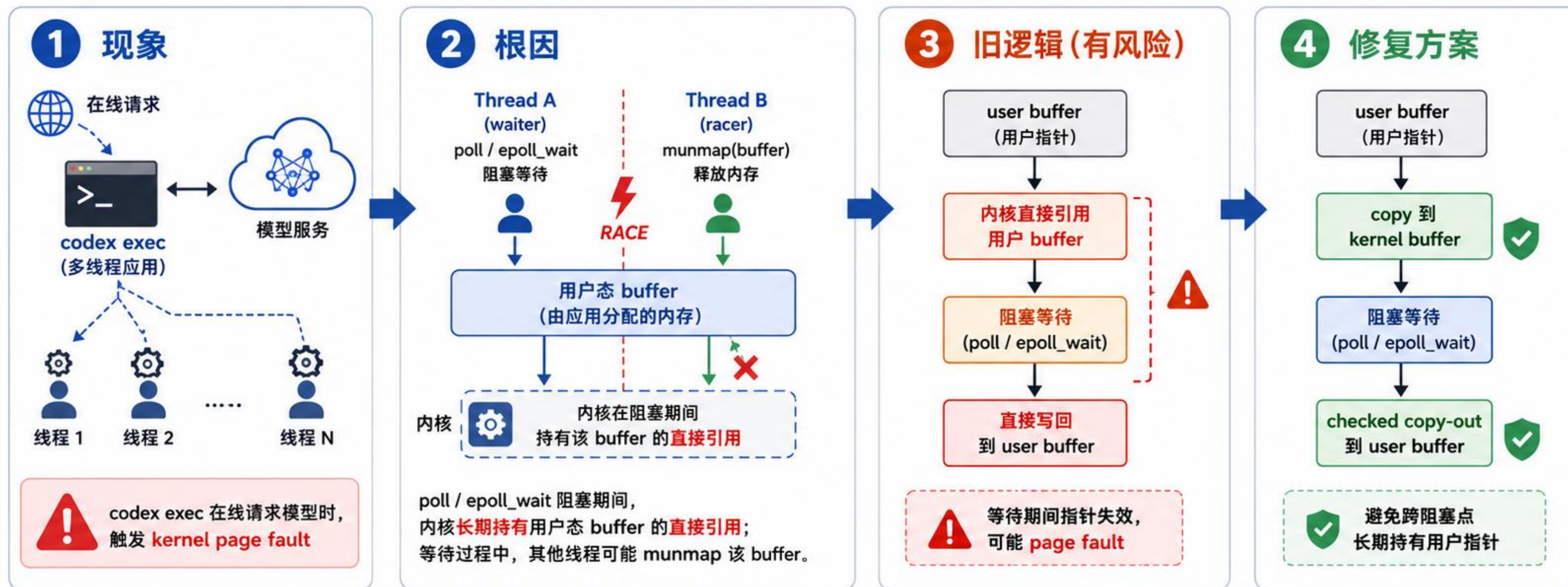
Codex CLI 暴露出的真实系统压力

Codex CLI



Exp4: Codex CLI on StarryOS

poll/epoll 用户缓冲区问题
(复杂应用暴露用户指针TOCTOU)



总结

用户指针不能**跨阻塞点**长期持有；真实多线程应用会触发 **race**，而这类问题往往难被小测试覆盖。

Exp4: Codex CLI on StarryOS

```
target/debug/tg-xtask - os_biglabB_task2 [WSL: Ubuntu-24.04] - Visual Studio Code

PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PWD=/

Use apk to install packages.

> "$(cat /tmp/tgoskits-syscall-hunt-prompt.txt)"
OpenAI Codex v0.115.0 (research preview)
-----
workdir: /tmp/tgoskits-syscall-hunt/repo
model: gpt-5.4
provider: openai
approval: never
sandbox: danger-full-access
reasoning effort: none
reasoning summaries: none
session id: 019e2629-66af-7801-aaaa-6ea9473dd605
-----
user
You are running inside a StarryOS x86_64 QEMU guest. The working directory is a real clone of the TGO SKits repository.

Task: find and, if practical, fix one small and well-scoped syscall or kernel semantic bug.

Rules:
- Treat Linux behavior as the reference.
- Prefer one small target from filesystem/directory semantics, process wait/exit semantics, or memory mapping semantics.
- Handle only one clear issue in this run. Do not do a broad refactor.
- First read `AGENTS.md`, `test-suite/starryos/GUIDE.md`, and the relevant StarryOS kernel/test-suite code.
- First explain the minimal Linux-vs-StarryOS differential idea, then decide whether to write a test.
- If it is safe to edit, prefer adding or updating one minimal regression test and one minimal fix.
- If time, tools, or environment limits prevent a safe fix, do not force a patch. Instead report the candidate bug, evidence, relevant files, and the next minimal va.
- Do not read, print, or copy any auth.json.

Use necessary read-only commands such as `rg`, `sed`, and `git status`. If you edit files, only edit files directly related to the selected issue.

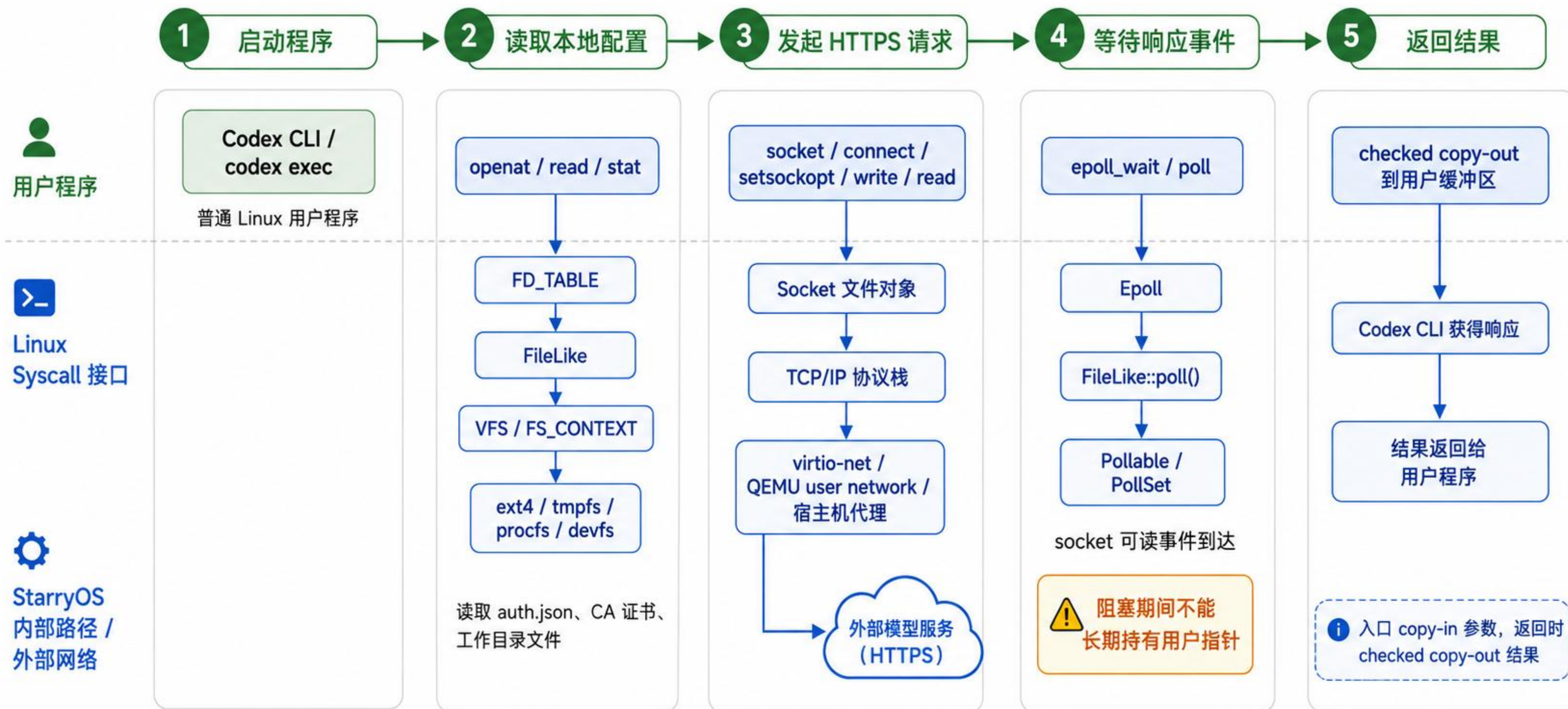
Your final answer must be in Simplified Chinese and must include:
- `STARRY_TGOSKITS_SYSCALL_HUNT_DONE`
- selected syscall/semantic target
- bug hypothesis or confirmed issue
- key files inspected
- if files changed, changed files and validation commands; if no files changed, the next minimal validation plan
mcp startup: no servers
```


Starry 整体架构与功能模块



Codex CLI 一次在线模型请求

用一条真实请求路径，理解 StarryOS 中关键模块如何协同



➤ 复用 ArceOS 基础能力

StarryOS 建立在 ArceOS 组件之上，复用调度、文件系统、网络栈等基础设施，开发重点可以放在 Linux 兼容层。

➤ 模块划分较清晰

内核按 syscall / task / mm / file / net 等模块组织，基本对应 OS 的核心组成部分。

➤ 面向真实 Linux 应用兼容

通过 BusyBox、curl、git、Codex CLI 等真实程序反向测试内核，而不是只验证简单 syscall。

➤ 统一的文件描述符抽象

把普通文件、目录、socket、pipe、epoll 等统一放进 fd 表，符合 Linux “一切皆文件”的接口风格。

➤ 兼容功能是按失败点逐步补齐的，尚未沉淀成统一的 ABI/状态治理层

StarryOS 已经能通过很多真实应用测试，但不少 Linux 兼容能力仍是“哪个应用失败就补哪个路径”。这样短期很有效，长期会导致 ABI 规则、系统状态和生命周期约束分散在各个 syscall 或接口实现里。

➤ 例：ABI 错误语义分散

linkat/renameat2 中，非法 flags 和坏用户指针同时出现时，Linux 对错误优先级有明确行为。如果每个 syscall 自己决定先读指针还是先校验 flags，就容易和 Linux 不一致。

➤ 改进方向

将 StarryOS 从“按应用补兼容路径”推进到“统一架构治理”：syscall 层形成清晰的 ABI wrapper，集中处理参数校验、用户指针和 errno；进程、网络、文件系统等模块也可以做类似处理；跨模块不变量用文档或回归测试固定下来。

未来课程和实验改进方向

- 未来的OS课程该怎么上？
 - ✓ 或许自学+讨论？ 参考某位清华物理老师授课
- 未来的OS实验如何改进？
 - ✓ 从0搭建一个极小OS如何！ ？

➤ 完成情况

- ✓ 完成 ArceOS/TGOSKits 基础实验，建立系统分层理解
- ✓ 搭建 Codex 闭环 Pipeline
- ✓ Exp2: 8 个 syscall 兼容性 PR 合入
- ✓ Exp3: 7 个 BusyBox 兼容性 PR 合入
- ✓ Exp4: Codex CLI 在 StarryOS x86_64 QEMU 中运行
- ✓ 实现最小 self-evolve demo:

StarryOS 运行 Codex, Codex 反过来阅读和修改 StarryOS/TGOSKits 代码

➤ 对 Starry OS 内核架构的理解

➤ 对未来课程和实验改进方向的建议

未来的AI会变成什么样？

未来的research应该是什么样？

未来的AI会变成什么样？

未来的research应该是什么样？

未来，人人将成为reviewer！

人只应该定义“意义”和“评价”。



祝愿未来的 OS BIGLAB 越来越好

敬请老师和助教批评指正

OS BIGLAB 总结报告 | 2026.04